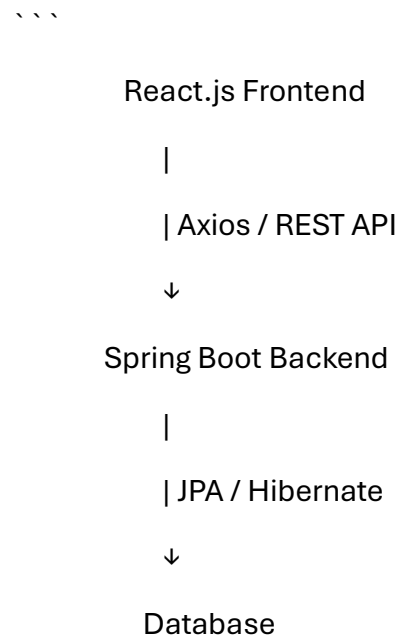


1. Project Overview

> "I developed an Employee Management System where users can perform CRUD operations on employee records. The frontend is developed using React.js, and it communicates with a Spring Boot REST API using Axios. The application allows users to log in, view employees, add new employees, update existing employees, and delete employees. I also implemented client-side validation, protected routes, and pagination."

The basic architecture is:



2. Technologies Used

Frontend

- React.js
- React Router
- Axios
- Bootstrap
- JavaScript
- `react-simple-typewriter`

Backend

- Java
- Spring Boot
- Spring Data JPA
- Hibernate
- REST APIs

Database

Depending on your backend setup, you're using a relational database such as MySQL.

3. Main Features

Your project currently has these major features:

1. Login

User first goes through the login page.

You have:

```
```jsx
<Route path="/login" element={<Login />} />
```
```

After successful login, you store something like:

```
```javascript
localStorage.setItem("logged", "true");
```
```

Then the user can access protected pages.

2. Employee List

The main page displays employees in a table.

For example:

```
```
```

```

ID | Name | DOJ | Department | Designation | Action
```

```

```

```
1 | Rahul | ... | IT | Developer | Update Delete
```

2 | Amit | ... | HR | Manager | Update Delete

---

```

The frontend calls:

```javascript

EmployeeService.getAllEmployees()

```

which makes:

```http

GET http://localhost:9191/api/v1/employee

```

The backend returns employee data as JSON.

4. Employee Object

Your employee structure looks approximately like:

```json

{

  "id": 1,

  "name": "Sourabh",

```
"doj": "25-09-2026",
"dept": {
 "deptName": "IT",
 "designation": "Developer"
}
}
```

So you have an employee and a nested department object.

---

## 5. Create Employee

When the user clicks:

...

Add Employee

...

React navigates to:

...

/add-emp

...

Your `CreateEmployee.jsx` maintains the form data using React state:

```
`` `jsx
const [employees, setEmployees] = useState({
 name: "",
 doj: "",
 dept: {
 deptName: "",
 designation: ""
 }
});
` `
```

The user enters:

```
` ` `
Name
DOJ
Department
Designation
` ` `
```

Then `handleSubmit()` is executed.

```
`` `jsx
EmployeeService.addEmployee(employeeData)
` ` `
```

This ultimately performs:

```
```http
POST /api/v1/employee
```
```

with JSON data.

---

## 6. Form Validation

You also implemented client-side validation.

For example:

```
```jsx
if (!employees.name) {
  formErrors.name = "Name is mandatory";
  isValid = false;
}
```
```

Similarly:

```
```jsx
if (!employees.doj)
```
```

and:

```
` `` `jsx
if (!employees.dept.deptName)
` `` `
```

and:

```
` `` `jsx
if (!employees.dept.designation)
` `` `
```

So the form doesn't submit if mandatory fields are missing.

This is a good point to mention in an interview.

---

## 7. Update Employee

When the user clicks:

```
` `` `
```

Update

```
` `` `
```

you generate a URL such as:

```
` `` `
```



```
/update-emp/5
```

```
...
```

The `5` is the employee ID.

Your route is:

```
```jsx
```

```
<Route
```

```
  path="/update-emp/:id"
```

```
  element={
```

```
    <PrivateRoute>
```

```
      <UpdateEmployee />
```

```
    </PrivateRoute>
```

```
  }
```

```
```
```

Inside `UpdateEmployee.jsx`:

```
```jsx
```

```
const { id } = useParams();
```

```
...
```

React Router extracts:

```
...
```

```
id = 5
```

```
    ` ` `
```

Then you call:

```
    ` ` `jsx
```

```
    EmployeeService.getEmployeeById(id)
```

```
    ` ` `
```

which sends:

```
    ` ` `http
```

```
    GET /api/v1/employee/5
```

```
    ` ` `
```

The existing employee information is loaded into the form.

```
    ---
```

8. Updating the Data

When the user clicks Submit:

```
    ` ` `jsx
```

```
    EmployeeService.updateEmployee(id, updateEmployee)
```

```
    ` ` `
```

Your service performs:

```
```http
```

```
PUT /api/v1/employee/5
```

```
```
```

The backend updates the employee.

Then:

```
```jsx
```

```
navigate("/");
```

```
```
```

takes the user back to the employee list.

9. Delete Employee

You implemented:

```
```jsx
```

```
const deleteEmployee = (id) => {
```

```
 EmployeeService.deleteEmployee(id)
```

```
 .then(() => {
```

```
 loadEmployees();
```

```
 });
```

```
};
```

```
```
```

The service calls:

```
```http
DELETE /api/v1/employee/{id}
```
```

For example:

```
```http
DELETE /api/v1/employee/5
```
```

After deletion, you call:

```
```jsx
loadEmployees();
```
```

so the latest employee list is displayed.

10. Axios Service Layer

One important design decision in your project is that you don't directly write Axios calls everywhere.

You created:

```
```
```

services/

EmployeeService.jsx

```
```
```

Inside:

```
```jsx
```

```
class EmployeeService {
```

```
 addEmployee(employee) {
```

```
 return axios.post(EMPLOYEE_API, employee);
```

```
 }
```

```
 getAllEmployees() {
```

```
 return axios.get(EMPLOYEE_API);
```

```
 }
```

```
 getEmployeeById(employeeId) {
```

```
 return axios.get(EMPLOYEE_API + '/' + employeeId);
```

```
 }
```

```
 updateEmployee(employeeId, employee) {
```

```
 return axios.put(
```

```
 EMPLOYEE_API + '/' + employeeId,
```

```
 employee
```

```

);
 }

 deleteEmployee(employeeId) {
 return axios.delete(
 EMPLOYEE_API + '/' + employeeId
);
 }
}
...

```

This is good because it separates:

...

UI logic

↓

Service/API logic

↓

Backend

...

Instead of putting all API calls directly inside your components.

---

## 11. React Routing

Your `App.js` controls navigation.

You have:

```

/login

/

/add-emp

/update-emp/:id

```

Conceptually:

```

Login

|

↓

Employee List

|

├— Add Employee

|

├— Update Employee

|

└— Delete Employee

```

---

12. PrivateRoute

You created:

```
```jsx
function PrivateRoute({ children }) {
  return localStorage.getItem("logged")
    ? children
    : <Navigate to="/login" />;
}
```
```

This means:

```
```
Is logged in?
|
YES|  NO
|  |
↓  ↓
Page  Login
```
```

For example, if somebody tries:

```
```
http://localhost:5173/
```
```



without being logged in, they are redirected to:

```
...

/login
...
```

This is frontend route protection.

One important interview point: this alone is not complete security. The backend should also authenticate/authorize API requests.

---

### 13. Pagination

You also implemented pagination.

You have:

```
```jsx  
const recordsPerPage = 3;  
...
```

So only three employees appear on one page.

You calculate:

```
```jsx
```

```
const indexOfLastEmployee =
 currentPage * recordsPerPage;
```

```
const indexOfFirstEmployee =
 indexOfLastEmployee - recordsPerPage;
` ``
```

Then:

```
` `` `jsx
const currentEmployees = employee.slice(
 indexOfFirstEmployee,
 indexOfLastEmployee
);
` ``
```

For example, with 10 employees:

```
` ``
```

Page 1 → Employee 1-3

Page 2 → Employee 4-6

Page 3 → Employee 7-9

Page 4 → Employee 10

```
` ``
```

You also have:

```
` ``
```

[Previous](#)

[1](#) [2](#) [3](#) [4](#)

[Next](#)

...

---

## 14. Overall Request Flow

This is probably the most important thing to understand for your interview.

Suppose the user clicks Add Employee.

### Step 1

React displays:

...

CreateEmployee.jsx

...

### Step 2

User enters:

...

Name

DOJ

Department

Designation

...

Step 3

React validates the form.

Step 4

`CreateEmployee.jsx` calls:

```
```javascript
```

```
EmployeeService.addEmployee(employeeData)
```

```
```
```

Step 5

Axios sends:

```
```http
```

```
POST /api/v1/employee
```

```
```
```

Step 6

Spring Boot Controller receives the request.

Conceptually:

```
```java
@PostMapping
public Employee addEmployee(@RequestBody Employee employee) {
    return employeeService.addEmployee(employee);
}
```
```

Step 7

Service layer processes the business logic.

...

Controller

↓

Service

↓

Repository

↓

Database

...

Step 8

Database stores the employee.

Step 9

Backend returns the response.

## Step 10

React executes:

```
```jsx
navigate("/");
```
```

and displays the updated employee list.

---

## 15. Backend Architecture

Your backend should ideally follow:

```

Controller

↓

Service

↓

Repository

↓

Entity

↓

Database

...

For example:

...

EmployeeController

↓

EmployeeService

↓

EmployeeRepository

↓

Employee Entity

↓

MySQL

...

Controller

Handles HTTP requests.

...

GET

POST

PUT

DELETE

...